

Simulação Concorrente de um Centro de Distribuição Automatizado (IDP)

Samuel Abrão¹, Andreia Tiveron¹, João Genaro¹

¹Instituto Brasileiro de Ensino, Desenvolvimento e Pesquisa (IDP)
Asa Norte – Brasília – DF – Brasil

samuelabrao2006@gmail.com, andreiabtiveron@gmail.com,
genastrb@gmail.com

Abstract. *This report describes a concurrent simulation, written in C with POSIX threads, of an automated distribution center. Collector and deliverer robots share a grid map, pickup stations and a bounded conveyor belt, moving one cell at a time under the physical constraint that two entities never occupy the same cell. We detail the thread organization, the shared resources, the identified critical sections, the synchronization mechanisms (fine-grained mutexes and a condition variable) and the main design decisions, followed by the build and run guide for Linux (Ubuntu 24.04).*

Resumo. *Este documento descreve uma simulação concorrente, escrita em C com threads POSIX, de um centro de distribuição automatizado. Robôs coletores e entregadores compartilham um mapa em grade, estações de coleta e uma esteira transportadora limitada, movendo-se célula a célula sob a restrição física de que duas entidades nunca ocupam a mesma célula. Detalham-se a organização das threads, os recursos compartilhados, as seções críticas identificadas, os mecanismos de sincronização (mutexes de granularidade fina e uma variável de condição) e as principais decisões de projeto, seguidos do guia de compilação e execução em Linux (Ubuntu 24.04).*

1. Introdução

A simulação reproduz o centro de distribuição da IDP, dividido em três regiões: a **área de coleta**, com estações geradoras de pacotes (pontos P); a **esteira transportadora**, ligando a entrada (in) à saída (out); e a **área de expedição**, com pontos de despacho (D). O fluxo de um pacote é: o *gerador* o cria em uma estação P; um *robô coletor* o transporta até in; a *esteira* o desloca uma posição por unidade de tempo até out; e um *robô entregador* o retira e o leva a um ponto D. A restrição física central é que *duas entidades nunca ocupam a mesma célula do mapa*. A simulação termina quando a quantidade de pacotes do cenário é entregue, ou quando o usuário pressiona q.

O sistema foi escrito em C com pthreads [1] e interface em ncurses [2]. Seguindo a recomendação do enunciado, primeiro foi construída e validada uma versão sequencial e só então introduziu-se a concorrência. O código é organizado em módulos independentes que correspondem aos elementos exigidos: *gerador* (Gerador de Pacotes), *coletor* (Robôs Coletores), *esteira* (Esteira Transportadora), *entregador* (Robôs Entregadores) e *interface* (Interface); os módulos *mapa* e *estacao* implementam os recursos compartilhados de base. O mapa é representado por duas matrizes:

uma de tipos de célula (livre, parede, estação P, ponto D) e uma de ocupação; cada pacote carrega um estado (*aguardando, transportado, na esteira, entregue*). São oferecidos três cenários estáticos, selecionáveis por argumento de linha de comando ou por menu (Tabela 1).

Tabela 1. Parâmetros dos três cenários estáticos.

Parâmetro	0 – Básico	1 – Intermediário	2 – Estresse
Mapa (largura $\times$ altura)	20 $\times$ 10	30 $\times$ 15	40 $\times$ 20
Robôs coletores	2	4	6
Robôs entregadores	2	3	5
Estações P	2	3	4
Pontos D	2	3	4
Tamanho da esteira	8	12	20
Total de pacotes	20	50	100

2. Organização das Threads

Cada entidade da simulação é uma *thread* POSIX independente, criada com `pthread_create` e finalizada com `pthread_join` (nenhuma *thread* é *detached*). A *thread* principal não modela uma entidade: ela desenha a interface (é a única a chamar `ncurses`), acompanha o critério de término e faz o `join` das demais. Cada *thread* de entidade executa um laço que dorme por um passo de tempo (`TICK_US`, 100 ms; coletores e entregadores usam metade do intervalo) e verifica, a cada iteração, uma *flag* global de encerramento.

- **Gerador** (1 *thread*): a cada passo cria um pacote em uma estação P, em *round-robin* com *liveness* (busca outra estação com espaço se a atual estiver cheia);
- **Esteira** (1 *thread*): a cada passo avança os pacotes uma posição rumo a `out`, se a posição seguinte estiver livre;
- **Coletores** (N *threads*): autômato de três estados — `OCIOSO` (procura estação com pacote), `INDO_ESTACAO` (desloca-se e coleta) e `INDO_ESTEIRA` (leva o pacote até `in` e o insere; se `in` estiver ocupada, aguarda);
- **Entregadores** (M *threads*): autômato de três estados — `OCIOSO` (espera pacote em `out`), `INDO_ESTEIRA` (vai a `out` e retira) e `INDO_DESPACHO` (leva o pacote a um ponto D e o contabiliza).

O total é $2 + N + M$ *threads* de entidades mais a principal (por exemplo, seis *threads* de entidades no cenário básico). O encerramento é coordenado: uma *flag* booleana global, protegida por `mutex`, é consultada por cada *thread* a cada iteração e sinalizada pela principal quando todos os pacotes são entregues ou o usuário tecla `q`; em seguida, todas as *threads* são unidas com `pthread_join`.

3. Recursos Compartilhados

Cada recurso possui trava própria, de granularidade fina, em vez de um único *lock* global, permitindo progresso paralelo. A Tabela 2 resume o mapeamento recurso $\rightarrow$ mecanismo.

A ocupação do mapa é o recurso mais disputado: todo robô a lê e a escreve ao se mover. As estações são filas FIFO acessadas pelo gerador (que enfileira) e pelos coletores (que desenfileiram). A esteira é um *buffer* limitado de uma posição por célula. Os

Tabela 2. Recursos compartilhados e mecanismos de sincronização.

Recurso	Estrutura	Mecanismo
Ocupação das células do mapa	Mapa.ocupada	Mapa.mutex (global do mapa)
Estações de coleta (filas FIFO)	Estacao.fila	um Estacao.mutex por estação
Posições da esteira (in..out)	Esteira.posicoes	Esteira.mutex + Esteira.cond
Pontos de despacho D	células do mapa	via Mapa.mutex
Contadores/estatísticas	Estatisticas	Estatisticas.mutex
Flag de encerramento	variável global	mutex_encerrar

pontos D não têm trava própria: são células do mapa e sua ocupação é coordenada pelo Mapa.mutex. Os contadores de pacotes aguardando, na esteira e entregues, além do tempo de execução, são atualizados por várias *threads* e lidos pela interface.

4. Seções Críticas

As seções críticas identificadas e protegidas são:

- **Ocupar uma célula (*test-and-set*).** Verificar que a célula de destino está livre e marcá-la como ocupada precisa ser *atômico*; caso contrário, dois robôs poderiam ocupar a mesma célula. A verificação e a marcação ocorrem sob Mapa.mutex, e o passo do robô libera a célula de origem e ocupa a de destino dentro do mesmo *lock*, preservando a invariante física.
- **Fila da estação.** Enfileirar (gerador) e desenfileirar (coletores) ocorrem sob Estacao.mutex; vários coletores podem disputar a mesma fila simultaneamente.
- **Movimentação de pacotes na esteira.** Inserir em in, avançar todas as posições (varrendo de out para in para não mover um pacote duas casas no mesmo passo) e retirar de out ocorrem sob Esteira.mutex; as consultas (saída ocupada, total de pacotes) também travam, garantindo leitura consistente.
- **Contadores globais.** Incrementos e decrementos dos pacotes aguardando, na esteira e entregues ocorrem sob Estatisticas.mutex.
- **Leitura da interface.** Cada parte mutável do quadro é copiada (*snapshot*) sob o mutex do recurso correspondente antes de ser desenhada, de modo que a tela nunca exibe um estado inconsistente.

5. Mecanismos de Sincronização

O mecanismo básico é a exclusão mútua com pthread_mutex_t, aplicada com granularidade fina: um mutex por recurso. A esteira mantém, além do mutex, uma variável de condição (Esteira.cond), sinalizada com pthread_cond_broadcast a cada mudança de estado (inserção, avanço e retirada), pronta para suportar espera bloqueante; no ritmo atual, as entidades reavaliam o estado de forma temporizada (usleep por passo), sempre lendo o estado protegido pelo respectivo mutex. Cada pthread_mutex_init/pthread_cond_init possui o _destroy correspondente antes do término (a flag de encerramento usa inicializador estático), e cada pthread_create tem seu pthread_join, evitando vazamento de recursos e threads pendentes.

6. Decisões de Projeto

- **Desenvolvimento incremental.** A lógica de cada entidade foi escrita como uma função de *um passo* do seu autômato e validada de forma sequencial; a concorrência foi introduzida depois, reutilizando o mesmo passo como corpo do laço de cada *thread*.
- **Granularidade fina de travas.** Um *mutex* por recurso, em vez de um *lock* global, para permitir progresso paralelo e reduzir contenção.
- **Navegação sem *deadlock*.** Heurística gulosa (distância de Manhattan) com desvio lateral quando o caminho está bloqueado; robôs ociosos recuam a uma base para não obstruir *in/out*, evitando o travamento de dois robôs frente a frente no corredor.
- **Memória previsível.** Pacotes e robôs vivem em vetores de tamanho fixo (`MAX_*`), sem `malloc` por entidade.
- ***Backpressure*.** Filas de coleta limitadas e gerador em *round-robin* com *liveness*, que procura outra estação com espaço em vez de desistir quando a primeira está cheia.
- **Verificação.** Testes unitários por módulo (`make test`) e detecção de condições de corrida com o *ThreadSanitizer* (`make test-tsan`).

7. Guia de Compilação e Execução

Ambiente de referência: Linux (Ubuntu 24.04). Instalação das dependências (compilador, *make* e a biblioteca `ncurses`):

```
sudo apt install build-essential libncurses-dev
```

Compilação (`gcc` com `-Wall -Wextra -pthread` e `-lncurses`), execução e testes:

```
make                # gera o executável em build/idp
make run            # compila e executa (menu de cenários)
./build/idp 0       # executa diretamente o cenário 0, 1 ou 2
make test           # testes unitários dos módulos
make test-tsan      # teste de concorrência sob ThreadSanitizer
make clean          # remove artefatos de compilação
```

Em terminal interativo, a interface `ncurses` exibe o mapa, os robôs, a esteira e o painel (pacotes aguardando, na esteira, entregues e tempo de execução); tecle `q` para sair. Sem terminal interativo, o programa imprime um quadro final em texto com as mesmas informações, útil para validação automatizada.

Referências

- [1] TANENBAUM, A. S. *Sistemas Operacionais Modernos*. 3. ed. São Paulo: Pearson Prentice Hall, 2010.
- [2] FREE SOFTWARE FOUNDATION. *ncurses — A Free Software emulation of curses in System V Release 6.6*. Disponível em: <https://invisible-island.net/ncurses/announce.html>.